

**LAPORAN TUGAS PROYEK AKHIR
LEADERBOARD GAME**

**Disusun untuk Memenuhi Tugas Akhir Project-Based Learning (PBL)
Mata Kuliah Algoritma & Struktur Data (TPL2106)**



Disusun Oleh :

Kelompok 12

Kelas B

Muhammad Rezki Khaidir	J0403251127
Najla Septian Putri	J0403251010
Nayla Aqila Argia	J0403251142

**PROGRAM STUDI TEKNOLOGI REKAYASA PERANGKAT LUNAK
SEKOLAH VOKASI IPB UNIVERSITY**

2026

KATA PENGANTAR

Puji dan Tunga kehadiran Allah SWT. Yang telah memberikan Tunga-Nya sehingga penulis dapat menyelesaikan laporan tugas proyek akhir ini dengan judul “Leaderboard Game”. Adapun tujuan dari pembuatan laporan ini Tunga untuk memenuhi tugas mata kuliah Algoritma & Struktur Data.

Ucapan terima kasih ini penulis tujukan kepada berbagai pihak yang telah membantu dalam proses penyelesaian makalah ini, yaitu:

1. Dr. Ir. Aceng Hidayat, M.T., yang telah memberikan fasilitas dan kesempatan kepada kami untuk menimba ilmu.
2. Medhanita Dewi Renanti, S.Kom., M.Kom., yang telah memberikan dukungan serta motivasi dalam penyusunan laporan ini.
3. Dr. Shelve Nidya Neyman S.Kom., M.SI., selaku Dosen Kuliah Mata Kuliah Algoritma & Struktur Data yang telah memberikan bimbingan dan arahan materi.
4. Dr. Sofiyanti Indriasari S.Kom., M.Kom., selaku Dosen Praktikum Mata Kuliah Algoritma & Struktur Data yang telah membimbing kami dalam proses penyusunan.
5. Kedua orang tua kami, yang senantiasa memberikan doa, dan dukungan moral.
6. Rekan-rekan satu kelompok, atas kerja sama yang terjalin selama pengerjaan tugas ini.
7. Teman-teman satu kelas, yang telah memberikan semangat dan dukungan.

Dalam penulisan laporan ini penulis menyadari bahwa masih jauh dari kesempurnaan, untuk itu penulis mengharapkan kritik dan saran yang membangun untuk kebaikan di masa yang akan datang. Semoga laporan ini dapat berguna dan bermanfaat bagi yang memerlukannya terutama bagi penulis sendiri.

Bogor, 14 Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
BAB I. PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan Proyek.....	1
1.3 Ruang Lingkup.....	1
BAB II. ANALISIS DAN PERANCANGAN	2
2.1 Deskripsi Sistem	2
2.2 Struktur Data yang Digunakan.....	2
2.3 Algoritma yang Digunakan	3
2.4 Diagram Alir Program (Flowchart).....	3
2.5 Struktur Program	11
BAB III. IMPLEMENTASI PROGRAM	14
3.1 Tampilan Menu Utama.....	14
3.2 Fitur Create	15
3.3 Fitur Read.....	15
3.4 Fitur Update	15
3.5 Fitur Delete	16
3.6 Fitur Searching.....	17
3.7 Fitur Sorting.....	17
3.8 Penyimpanan Data (File Handling)	18
3.9 Validasi Input dan Error Handling	18
BAB IV. PENGUJIAN DAN ANALISIS.....	20
4.1 Skenario Pengujian	20
4.2 Kelebihan Sistem.....	21
4.3 Kekurangan Sistem	22
4.4 Keterbatasan Sistem	23
BAB V. KENDALA DAN SOLUSI.....	25
BAB VI. PEMBAGIAN TUGAS ANGGOTA	26
BAB VII. KESIMPULAN DAN SARAN	27
7.1 Kesimpulan	27
DAFTAR PUSTAKA.....	29
LAMPIRAN	30

BAB I. PENDAHULUAN

1.1 Latar Belakang

Pengembangan aplikasi permainan interaktif yang mendidik memerlukan rname manajemen data hasil skor yang andal dan terstruktur. Masalah yang sering dihadapi pada aplikasi game berbasis terminal berskala kecil rname hilangnya rnamen data skor pemain rname program dihentikan, serta tidak adanya visualisasi urutan pemain terbaik (leaderboard) yang kompetitif secara *real-time*.

Proyek ini dibuat untuk menyediakan rname berupa rname *Leaderboard Game Tebak Angka* yang dinamis. Sistem ini dirancang untuk merekam data skor pemain secara permanen menggunakan penanganan berkas eksternal (file handling), mengurutkan peringkat secara langsung, serta menyediakan fitur pencatatan log rnamen penghapusan data untuk menjaga keamanan operasional aplikasi.

1.2 Tujuan Proyek

1. Membuat aplikasi permainan tebak angka berbasis terminal (CLI) menggunakan rname pemrograman Python.
2. Mengimplementasikan konsep struktur data Array Dinamis (Python List) untuk penyimpanan utama dan struktur data Stack (Tumpukan) untuk rnamen perubahan data.
3. Menerapkan penanganan berkas (File Handling CSV) agar seluruh data skor pemain tersimpan secara permanen.
4. Mengembangkan logika algoritma pencarian linear (Linear Search) untuk mempermudah pelacakan skor pemain secara cepat.

1.3 Ruang Lingkup

Platform: CLI (Console / Terminal)

Bahasa Pemrograman: Python

Penyimpanan: File CSV (ZtugasAkhir/data.csv)

Fitur Utama: CRUD (Create Game Score, Read Leaderboard, Update Score, Delete Player Score), Searching (Cari nama pemain), dan Sorting (Urutan peringkat skor terbaik).

BAB II. ANALISIS DAN PERANCANGAN

2.1 Deskripsi Sistem

Aplikasi “Leaderboard Game Tebak Angka” merupakan aplikasi interaktif berbasis teks di mana pengguna dapat memainkan game matematika tebak angka. Sistem ini secara otomatis menghitung skor akhir pemain dan menyinkronkannya ke dalam database lokal berbentuk CSV. Pengguna dapat menggunakan menu navigasi utama untuk memanggil fungsi penambahan data skor baru, melihat tabel peringkat (leaderboard) teratas yang terurut menurun, melacak data pemain, memperbarui nilai skor, menghapus pemain dari tabel, hingga melihat tumpukan log aktivitas penghapusan.

2.2 Struktur Data yang Digunakan

Struktur Data	Fungsi
Python List (Array Dinamis)	Menampung seluruh objek data utama pemain (berupa kumpulan dictionary dengan pasangan key “nama” dan “skor”). Digunakan untuk pemrosesan iterasi data.
Stack (Tumpukan – LIFO)	Mengelola tumpukan history_stack untuk menyimpan log rnamen data pemain yang telah dihapus sementara waktu selama program dijalankan.
Dictionary	Menyimpan representasi entitas satu data pemain secara spesifik agar pemanggilan properti nama dan skor lebih terstruktur.

Alasan Pemilihan Struktur Data:

1. **List (Array Dinamis):** Dipilih karena sangat efisien untuk operasi penambahan data secara cepat menggunakan fungsi .append() tanpa perlu mendeklarasikan ukuran memori di awal, serta mempermudah pengolahan algoritma sorting bawaan.
2. **Stack:** Memakai prinsip LIFO (*Last In First Out*). Struktur ini sangat ideal untuk kasus pelacakan rnamen aktivitas atau sebagai dasar arsitektur fitur

pembatalan aksi (*undo*) di masa mendatang, karena operasi penghapusan terakhir akan berada di bagian paling atas.

2.3 Algoritma yang Digunakan

1. Searching (Linear dan Sequential Search):

Algoritma memeriksa setiap elemen di dalam list pemain satu per satu dari indeks awal hingga akhir. Hal ini sangat efektif untuk pencarian data teks berskala dinamis dan fleksibel (menggunakan manipulasi `.lower()` sehingga pencarian bersifat *case-insensitive*).

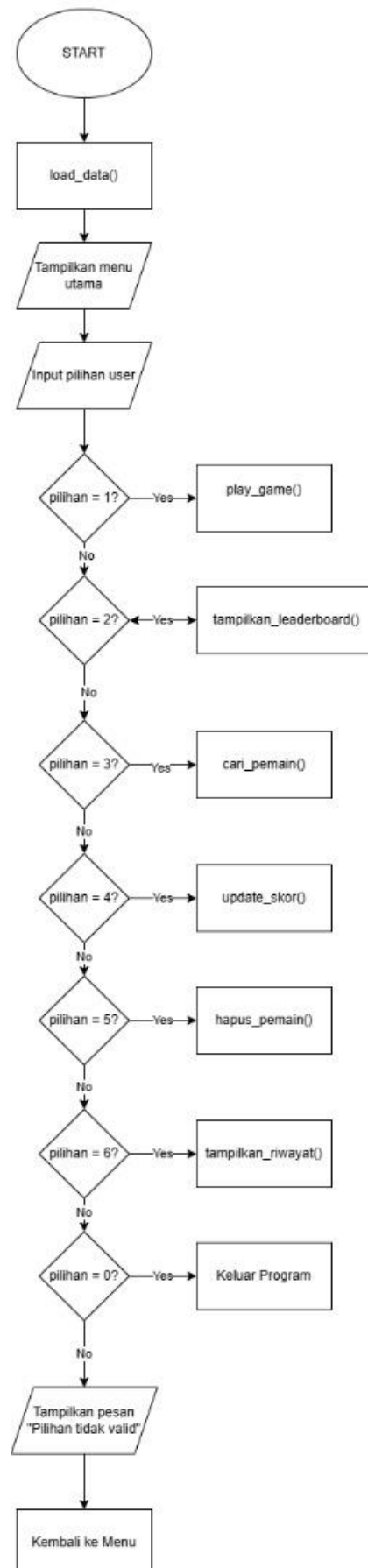
2. Sorting (Descending Sort dan Merge sort):

Digunakan untuk 3rnament urutan papan skor (leaderboard) secara menurun (*descending*) berdasarkan besaran nilai angka skor pemain. Ini memastikan bahwa pemain dengan pencapaian skor tertinggi selalu diposisikan di baris paling atas.

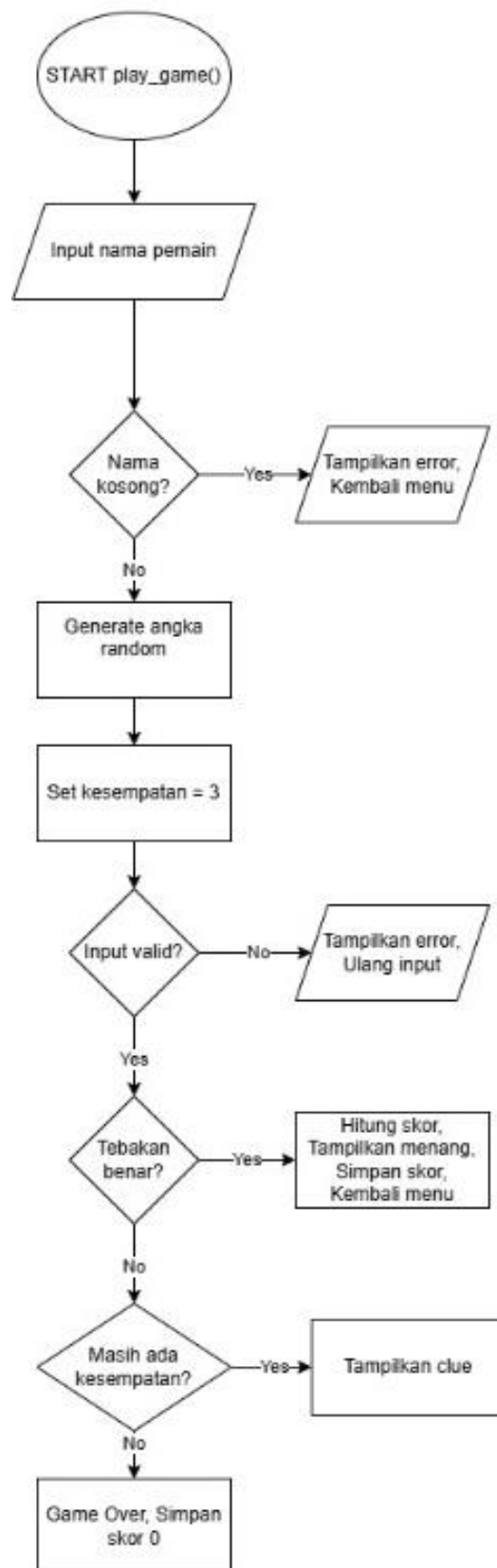
2.4 Diagram Alir Program (Flowchart)

Berikut merupakan representasi diagram alur logika dari sstruktur menu utama beserta jalannya fungsi pada aplikasi.
(halaman berikutnya)

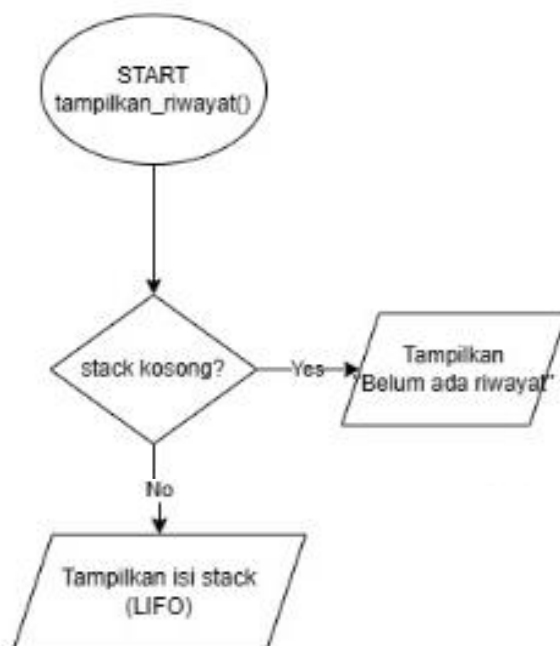
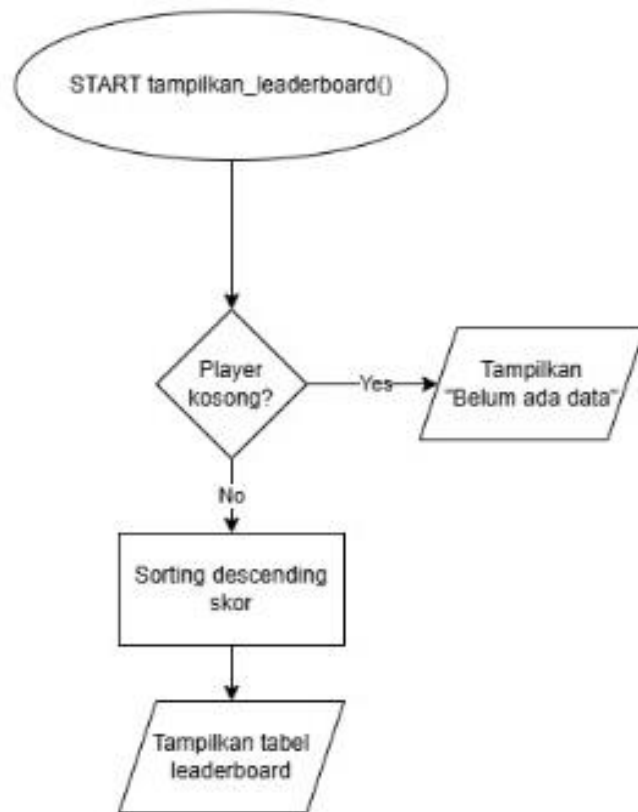
1. Menu Utama



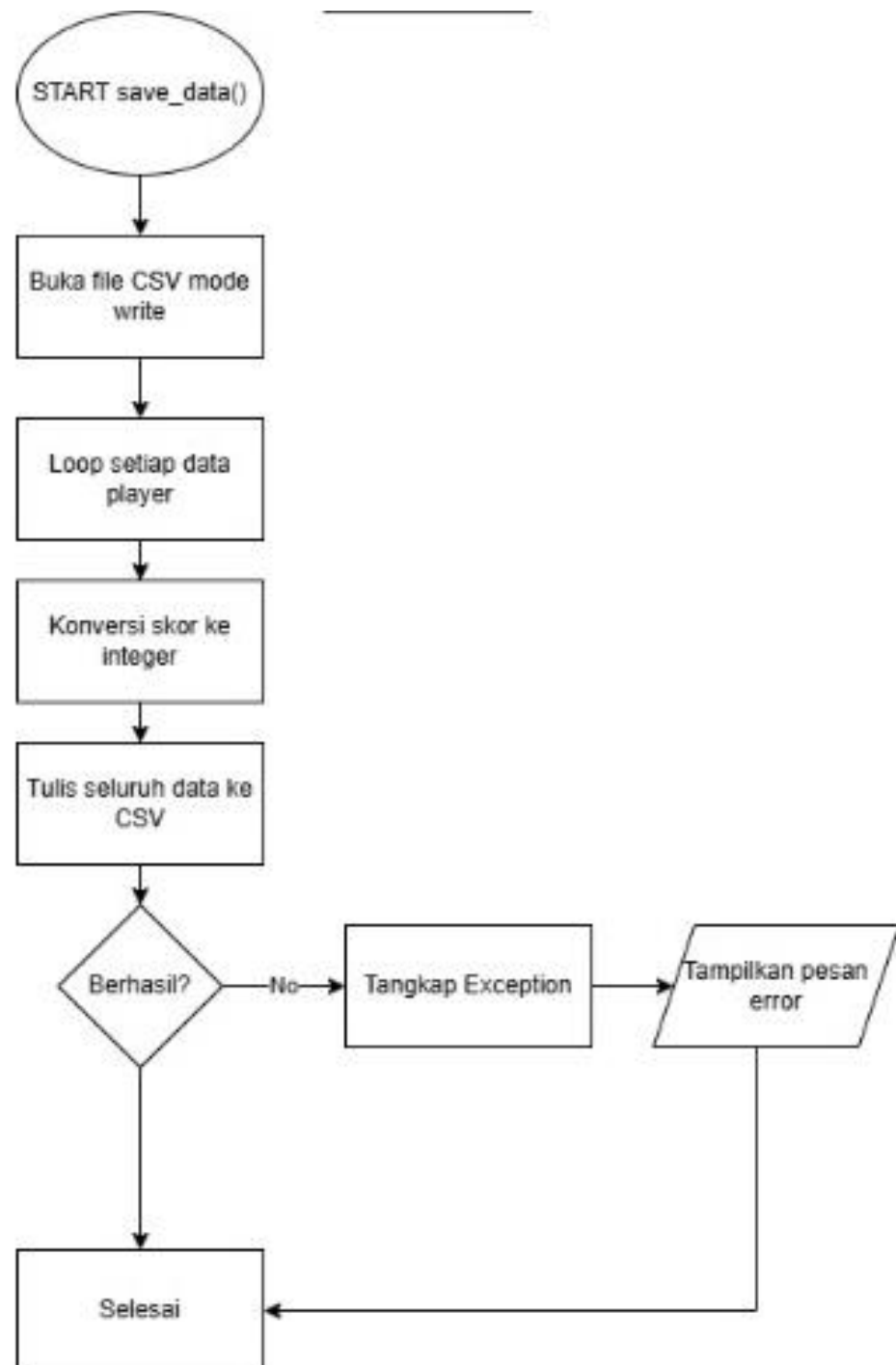
2. Fungsi play_game()



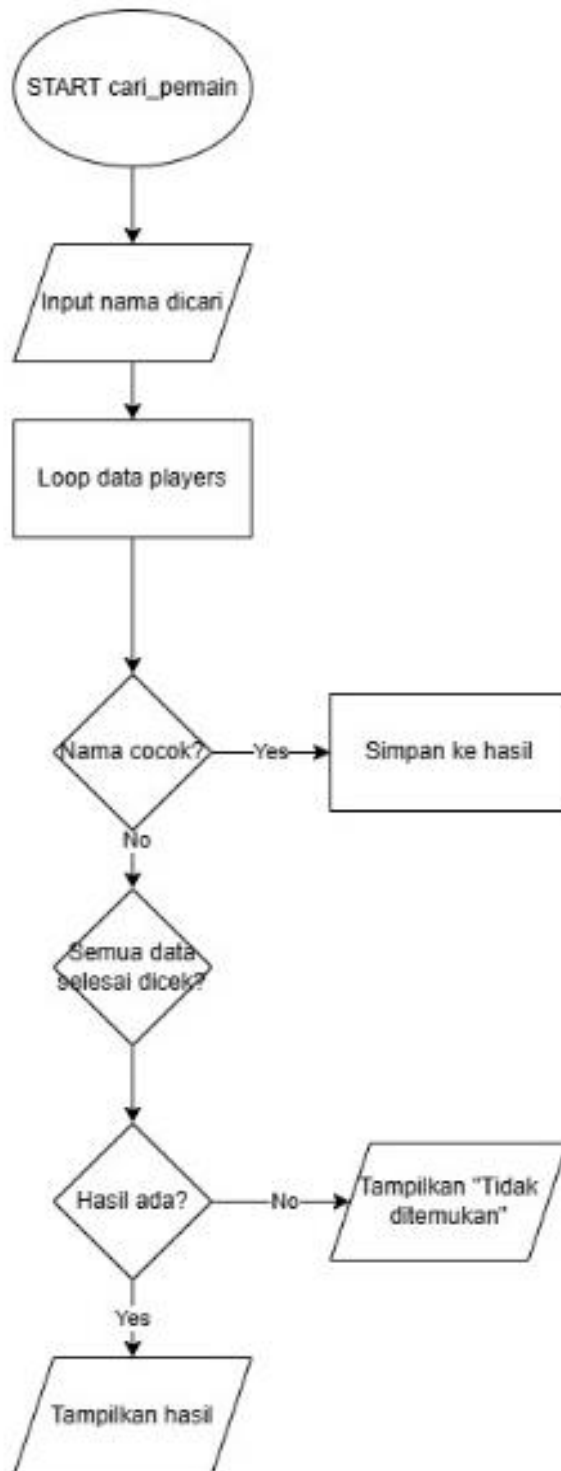
3. Fungsi tampilkan_leaderboard()



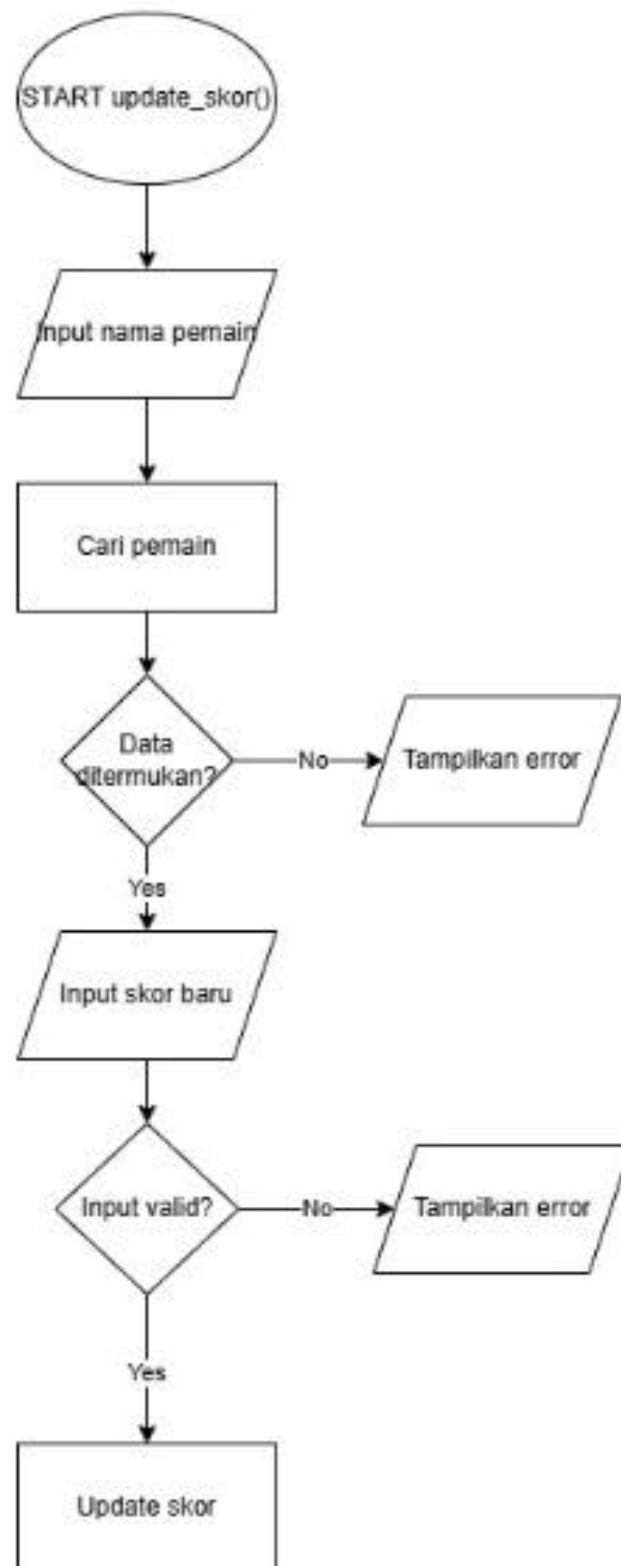
4. Fungsi save_data()



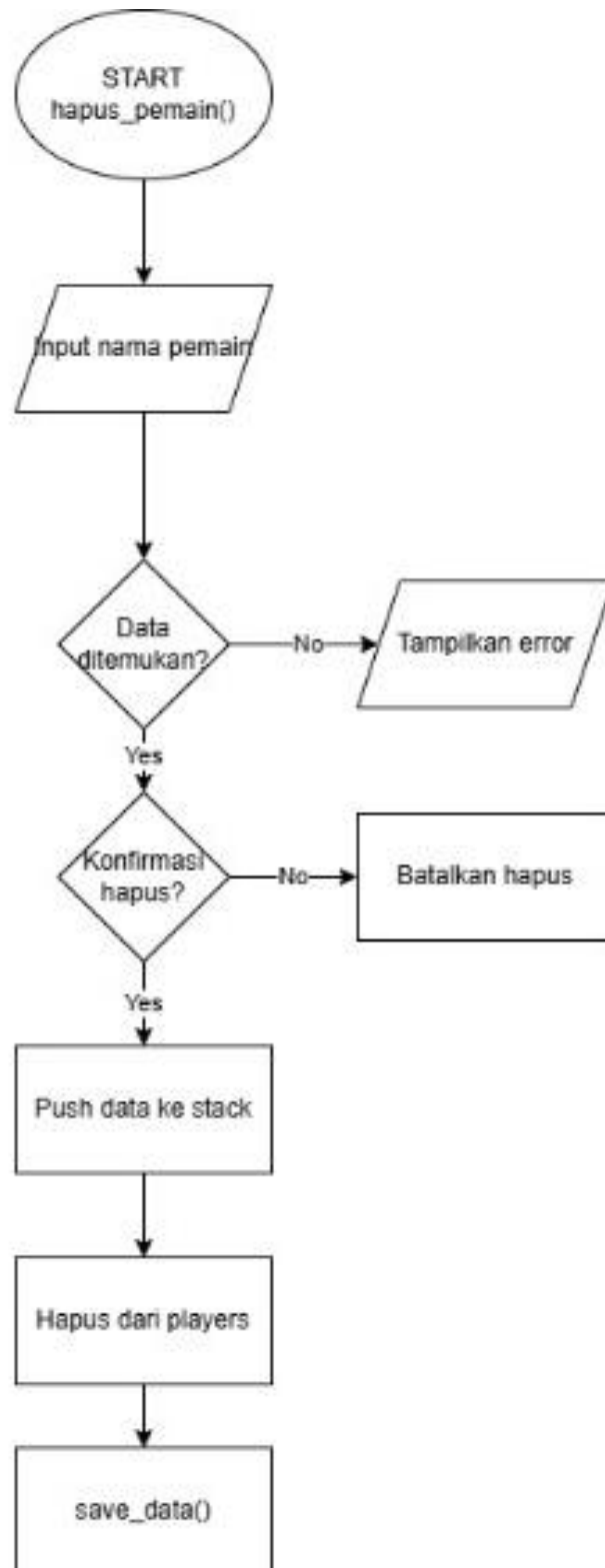
5. Fungsi cari_pemain()



6. Fungsi update_skor()



7. Fungsi hapus_pemain()



2.5 Struktur Program

Aplikasi “Leaderboard Game Tebak Angka” dirancang menggunakan pendekatan modular (fungsional) di mana setiap fitur utama diisolasi ke dalam fungsi-fungsi spesifik untuk mempermudah perawatan kode (*maintainability*) dan pembacaan alur kerja program.

Secara arsitektural, program ini terdiri dari satu berkas kode utama bernama `LEADERBOARD_GAME.py` dan satu berkas basis data bernama `data.csv` yang tersimpan di dalam folder bernama `ZtugasAkhir`.

Berikut ini rincian struktur fungsi-fungsi (modul) yang membentuk aplikasi ini:

4. Fungsi Manajemen File (*File Handling Module*)

- **`load_data()`**

Berfungsi untuk membaca data rekam skor yang tersimpan secara permanen di dalam berkas `ZtugasAkhir/data.csv`. Fungsi ini juga dilengkapi dengan validasi otomatis; jika direktori atau berkas belum terbentuk, fungsi akan membuat folder dan file baru beserta header kolomnya (Nama, Skor).

Parameter Input: Tidak ada.

Output / Return: list (berisi nama dictionary data pemain yang siap dimuat ke memori RAM).

- **`save_data(players)`**

Berfungsi untuk melakukan penulisan nama (*overwrite*) seluruh data terbaru dari memori RAM (nama list `players`) ke dalam file penyimpanan fisik `data.csv` menggunakan nama `csv.DictWriter`.

Parameter Input: `players` (list of dict).

Output / Return: Tidak ada (prosedur penyimpanan langsung ke *storage*).

2. Fungsi Inti Permainan (*Core Game Module*)

- **`play_game()`**

Mengatur logika permainan tebak angka interaktif. Fungsi ini menggunakan generator angka acak dari nama `random` untuk menghasilkan angka rahasia, membatasi kesempatan menebak pengguna (maksimal 3 kali), menghitung skor berdasarkan sisa kesempatan, dan secara otomatis memanggil fungsi `save_data()` untuk memperbarui database jika permainan selesai.

Parameter Input: Tidak ada.

Output / Return: Tidak ada (interaksi langsung via CLI).

3. Fungsi Manipulasi Struktur Data (*CRUD & Algorithm Module*)

- **tampilkan_leaderboard(players)**

Mengimplementasikan algoritma Sorting Descending (Timsort bawaan Python dengan parameter `key=lambda x: x['skor'], reverse=True`) dari **Insertion Sort dan Merge Sort** (Timsort bawaan Python dengan parameter `key=lambda x: x['skor'], reverse=True`). Fungsi ini akan mengurutkan susunan objek di dalam list berdasarkan skor terbesar ke terkecil, lalu mencetaknya ke terminal dalam format rnam.

Parameter Input: players (list).

Output / Return: Menampilkan visualisasi rnam urutan peringkat skor terbaik.

- **cari_pemain(players)**

Mengimplementasikan algoritma **Linear/Sequential Search**. Melakukan iterasi satu per satu untuk mencocokkan input nama yang dicari pengguna dengan data nama yang ada di dalam rnament list. Bersifat *case-insensitive* menggunakan fungsi bawaan `.lower()`.

Parameter Input: players (list).

Output / Return: Menampilkan detail data pemain jika ditemukan, atau memunculkan notifikasi bahwa data tidak ada.

- **update_skor(players)**

Berfungsi mencari data pemain lama terlebih dahulu, lalu melakukan pembaruan (*update*) nilai skor sesuai dengan input numerik baru dari user. Data yang diperbarui di memori akan langsung disinkronkan ke file CSV.

Parameter Input: players (list).

Output / Return: Tidak ada.

- **hapus_pemain(players, history_stack)**

Berfungsi untuk menghapus data pemain dari list utama players (*Delete*). Sebelum benar-benar dihapus dari database, data pemain tersebut dimasukkan terlebih dahulu ke dalam rnament history_stack menggunakan operasi `.append()` (representasi dari fungsi **Push** pada struktur data Stack).

Parameter Input: players (list), history_stack (list bertindak sebagai Stack).

Output / Return: Tidak ada.

- **tampilkan_riwayat_hapus(history_stack)**

Membaca isi tumpukan aktivitas log penghapusan. Fungsi ini menampilkan data dengan urutan terbalik (elemen terakhir yang dihapus akan tampil paling atas), sesuai dengan prinsip **LIFO (Last In First Out)** pada struktur data Stack.

Parameter Input: history_stack (list).

Output / Return: Menampilkan daftar log 13nmen penghapusan.

4. Kontrol Alur Utama (*Main Control Flow*)

- **main()**

Bertindak sebagai *driver function* atau jantung utama program yang pertama kali dieksekusi. Berisi perulangan tak terbatas (while True) untuk menampilkan menu navigasi CLI (Menu 1 hingga Menu 7 untuk keluar), menangkap pilihan input dari pengguna, serta melakukan percabangan kondisi (if-elif-else) untuk memanggil fungsi-fungsi spesifik di atas.

BAB III. IMPLEMENTASI PROGRAM

3.1 Tampilan Menu Utama

Lingkungan pengembangan dan operasional eksekusi program “Leaderboard Game Tebak Angka” dirancang agar bersifat lintas platform (*cross-platform*). Spesifikasi lingkungan perangkat lunak pendukung rname sebagai berikut:

- Sistem Operasi: Aplikasi diuji dan dapat berjalan optimal pada lingkungan CLI (*Command Line Interface*) baik di rname operasi Microsoft Windows, Linux distro apa pun, maupun macOS.
- Bahasa Pemrograman: Menggunakan ekosistem rname Python versi 3.x murni tanpa ketergantungan pada rnamen pihak ketiga (*third-party libraries*).
- Pustaka Tambahan (Built-in Libraries):
 1. csv: Digunakan sebagai fondasi utama manipulasi berkas (*file handling*) melalui kelas csv.DictReader untuk pembacaan database rnam secara tabular dan csv.DictWriter untuk pembaruan data secara rekursif.
 2. os: Digunakan untuk berinteraksi dengan rname direktori rnam rnament, menguji keberadaan *path* folder, dan membuat direktori secara otomatis jika belum tersedia di perangkat pengguna.
 3. random: Digunakan sebagai mesin pembangkit bilangan bulat acak pseudo-random (*pseudo-random number generator*) untuk memproduksi nilai rahasia dalam sesi *gameplay*.
 4. time : Mengatur waktu jeda menu.

Antarmuka menu utama dirancang secara visual menggunakan kode warna teks **ANSI Escape Codes** (PINK, YELLOW, CYAN, GREEN, RED, dan RESET). Fungsi menu() bertugas membangun komponen kosmetik antarmuka berupa kotak presisi menggunakan karakter dekoratif terminal (▬, =, ▮, ▮▮, ▮▮▮, ▮▮▮▮). Alur eksekusi menu dikendalikan secara terpusat oleh fungsi main() menggunakan struktur kontrol perulangan tak terbatas (while True) dan percabangan kondisi if-elif-else untuk menangkap opsi masukan dari pengguna (0 sampai 6).

3.2 Fitur Create

Proses pembuatan data baru (*Create*) diimplementasikan saat pengguna menyelesaikan permainan pada fungsi `play_game()`. Setelah pengguna menebak angka rahasia atau kehabisan kesempatan, program akan mengalkulasikan skor akhir. Alur pembuatan datanya adalah sebagai berikut:

1. Program memanggil fungsi sub-modul `simpan_skor(nama, skor)`.
2. Data nama dan skor dibungkus ke dalam objek *dictionary* dengan format literal `{"nama": nama, "skor": skor}`.
3. Objek baru tersebut dimasukkan ke dalam list global `players` via metode `.append()`.
4. Fungsi `save_data()` langsung dipicu untuk menyinkronkan data di memori RAM tersebut ke dalam file `15rnam CSV`.

3.3 Fitur Read

Fitur pembacaan data (*Read*) pada aplikasi ini terbagi menjadi dua fungsi spesifik, yaitu membaca daftar peringkat utama dan membaca tumpukan log penghapusan:

1. **Membaca Tabel Leaderboard:** Fungsi `tampilkan_leaderboard()` memproses isi list `players`. Program menampilkan data tersebut ke terminal dalam bentuk `rnam` terstruktur dengan manipulasi format spasi rata kiri (`:<12, :<25, :<10`). Juara pertama secara visual diberikan penanda khusus berupa emoji mahkota ().
2. **Membaca Riwayat Penghapusan:** Fungsi `tampilkan_riwayat()` membaca isi `rnam` `history_stack`. Untuk mematuhi prinsip struktur data **Stack (LIFO)**, program menggunakan fungsi `reversed(history_stack)` agar data yang paling terakhir dihapus diposisikan dan dibaca di urutan paling atas `rnam` `rnamen`.

3.4 Fitur Update

Fitur pembaruan data (*Update*) ditangani oleh fungsi `update_skor()`. Berbeda dengan sistem pencarian fleksibel, fitur ini menuntut ketepatan identitas data secara mutlak. Program melakukan perulangan sekuensial di dalam list `players`

untuk mencari nama yang dicocokkan menggunakan operator kesamaan linear (`==`).

Proses ini bersifat **case-sensitive** (peka terhadap huruf besar dan kecil). Jika pengguna ingin memperbarui skor pemain bernama “Najla”, maka input yang dimasukkan harus ditulis “Najla”, bukan “najla” ataupun “NAJLA”. Apabila nama yang dimasukkan tidak sama persis, rname akan menolak proses pembaruan dan menampilkan pesan galat: *“Pemain dengan nama ‘...’ tidak ditemukan. Update dibatalkan.”* Jika ditemukan kecocokan mutlak, rname pemain[‘skor’] akan ditimpa dengan nilai integer baru, lalu program secara otomatis memanggil prosedur `save_data()`.

3.5 Fitur Delete

Fitur penghapusan data (*Delete*) diimplementasikan pada fungsi `hapus_pemain()`. Fitur ini mengadopsi pencarian berbasis **case-sensitive** menggunakan ekspresi logika kondisional yang ketat:

```
if p["nama"] == nama:
```

Sistem mengharuskan masukan nama pemain diinput dengan susunan huruf besar dan kecil yang sama persis dengan yang ada di database. Ketika nama pemain ditemukan cocok secara absolut, program akan meminta konfirmasi 16 rname berupa input pilihan (y/n). Masukan konfirmasi dibaca menggunakan fungsi `.lower()` agar pengetikan huruf ‘Y’ maupun ‘y’ tetap dianggap valid.

Sebelum elemen dihapus dari list utama `players` menggunakan metode `.remove(p)`, program menjalankan fungsi **Push** ke dalam struktur data tumpukan log dengan baris perintah `history_stack.append(("Hapus", p.copy()))`. Penggunaan metode `.copy()` bertujuan untuk mengamankan rname data asli agar rname yang disimpan di dalam *stack* tidak ikut rusak akibat proses penghapusan di list utama. Setelah rname selesai, file CSV langsung diperbarui secara otomatis.

3.6 Fitur Searching

Mekanisme pelacakan data pemain pada fungsi `cari_pemain()` mengadopsi prinsip algoritma **Linear Search** (Pencarian Sekuensial) bersyarat *substring matching*. Berdasarkan arsitektur kode sumber aktual, pencarian ini diimplementasikan **tanpa** fungsi penyeragaman huruf `.lower()`, sehingga algoritma pencarian berjalan secara **case-sensitive**.

Program melakukan iterasi elemen demi elemen menggunakan perulangan `for` untuk mengevaluasi kondisi kecocokan string melalui operator keanggotaan `in`:

```
for pemain in players:
    if nama_dicari in pemain['nama']:
        hasil_pencarian.append(pemain)
```

Karena tidak adanya modul penyeragaman *case*, pencarian kata kunci “Ar” hanya akan berhasil menjaring nama pemain seperti “Arya” atau “Ardi”, namun akan melewatkan nama “rnam” atau “rezki” meskipun mengandung kombinasi huruf yang mirip secara fonetis. Setiap objek *dictionary* yang lolos dari validasi peka huruf ini akan ditampung sementara ke dalam array dinamis `hasil_pencarian` sebelum akhirnya diprint secara beruntun ke terminal.

3.7 Fitur Sorting

Proses pengurutan peringkat papan skor (*Sorting*) ditangani di dalam fungsi `tampilkan_leaderboard()`. Pengurutan dilakukan secara *out-of-place* (tidak merusak susunan asli list global `players` di latar belakang) memanfaatkan fungsi bawaan Python `sorted()`. Algoritma internal yang mengeksekusinya bernama **Timsort** yang bekerja dengan efisiensi waktu performa rata-rata senilai $O(n \log n)$.

Operasi pengurutan dikonfigurasi menggunakan fungsi `lambda` sebagai penunjuk parameter bobot nilai, serta dilengkapi instruksi pembalikan urutan agar menghasilkan peringkat menurun (**Descending Order**):

```
sorted_players = sorted(players, key=lambda x: x['skor'],
                        reverse=True)
```

Melalui parameter `key=lambda x: x['skor']`, algoritma membandingkan nilai numerik skor antarpemain dari yang terbesar hingga terkecil. Elemen yang telah terurut rapi di dalam `sorted_players` inilah yang kemudian dibaca oleh `name` untuk disajikan kepada user.

3.8 Penyimpanan Data (File Handling)

Manajemen penyimpanan data eksternal (*File Handling*) dipusatkan pada file teks terstruktur `ZtugasAkhir/data.csv`. Prosedur penanganan berkas ini dibagi ke dalam dua fungsi utama:

1. **Fungsi `load_data()` (Input):** Berfungsi mengosongkan memori sementara lewat `players.clear()` kemudian membuka file CSV dengan mode baca ("`r`"). Baris data dibaca menggunakan kelas objek `csv.DictReader` untuk dipindahkan ke list memori. Pada fase ini, program melakukan penataan ulang tipe data (*data type casting*) di mana nilai skor yang awalnya berupa string teks murni diubah dipaksa menjadi tipe numerik bulat melalui sintaksis "`skor`": `int(row["skor"])`.
2. **Fungsi `save_data()` (Output):** Berfungsi membuka file CSV dengan mode tulis ulang (`mode='w'`). Menggunakan objek `csv.DictWriter`, fungsi ini memanggil perintah `writer.writeheader()` untuk mencetak baris judul kolom [`'nama'`, `'skor'`]. Guna mengantisipasi kerusakan struktur dokumen, program melakukan perulangan untuk memastikan seluruh tipe data skor telah terkonversi ke bentuk integer (`int`) sebelum disimpan serentak melalui metode `writer.writerows(players)`.

3.9 Validasi Input dan Error Handling

Aplikasi ini menerapkan arsitektur pertahanan program (*Robustness*) yang sangat ketat melalui kombinasi pengujian kondisi logika dan isolasi kode menggunakan blok `try-except` untuk menangani kegagalan `name` (*runtime error*):

1. Proteksi Crash Input Non-Numerik (`ValueError`)

Potensi kerusakan terbesar pada aplikasi CLI muncul saat pengguna menginput karakter alfabet atau `name` pada menu yang membutuhkan angka.

Masalah ini diantisipasi menggunakan blok penangkap eksepsi `try ... except ValueError` di dalam fungsi `play_game()` (saat menebak angka) dan fungsi `update_skor()` (saat menginput skor baru). Jika terjadi *error* konversi data, program tidak akan berhenti paksa (*force close*), melainkan menampilkan peringatan teks berwarna merah dan mengalihkan alur `nama` ke menu utama `main`.

2. Proteksi Kegagalan Akses File (Exception Handling)

Pada fungsi `save_data()`, seluruh operasi penulisan file dibungkus dalam struktur pengaman `try ... except Exception as e`. Blok ini bertindak sebagai `nama` penyelamat jika berkas CSV gagal diakses akibat masalah `nama`, seperti file sedang dibuka oleh program Excel atau pembatasan hak akses (*permission denied*) pada `nama` operasi `nama` pengguna.

3. Validasi String Kosong (Validation Filter)

Pada permulaan permainan di fungsi `play_game()`, program menguji input `nama` menggunakan metode `.strip()` dan pemeriksaan kondisi `if player_name == ""`. Validasi ini berfungsi menolak pengguna yang sengaja atau tidak sengaja menginput karakter spasi kosong sebagai `nama` mereka.

4. Validasi Keberadaan Berkas Fisik

Di dalam fungsi `load_data()`, `nama` `os.path.exists()` dikombinasikan dengan perintah `os.makedirs()` untuk mendeteksi sekaligus membuat folder `ZtugasAkhir` dan file `data.csv` secara otomatis di awal eksekusi, sehingga terhindar dari `nama` kemunculan error fatal `FileNotFoundError`.

BAB IV. PENGUJIAN DAN ANALISIS

4.1 Skenario Pengujian

Tahap pengujian merupakan fase krusial dalam siklus pengembangan perangkat lunak yang bertujuan untuk memastikan bahwa seluruh modul dan fungsi yang telah diimplementasikan pada Bab III dapat beroperasi secara valid, stabil, dan sesuai dengan spesifikasi rancangan rname yang diharapkan. Pada aplikasi “Leaderboard Game Tebak Angka” ini, evaluasi difokuskan pada pengujian fungsionalitas utama (*functional testing*) dengan mengadopsi metode pengujian acuan kotak hitam (*Black-Box Testing*).

Melalui pendekatan *Black-Box Testing*, pengujian dilakukan secara murni dengan mengevaluasi performa antarmuka berbasis CLI, kegunaan menu navigasi, ketepatan algoritma pencarian linear dan pengurutan Timsort, serta keandalan rname penanganan galat (*error handling*) rname menerima masukan abnormal dari pengguna, tanpa memeriksa struktur internal baris kode secara detail. Rangkaian pengujian ini dirancang untuk mensimulasikan berbagai kondisi interaksi pengguna riil di terminal, termasuk pengujian sifat peka huruf (*case-sensitive*) pada fitur manipulasi data.

Seluruh rencana, target ekspektasi, serta hasil rname dari uji coba fungsionalitas rname aplikasi Kelompok 12 ini dirangkum secara terstruktur dalam matriks rname pengujian yang disajikan pada Tabel 4.1 di bawah ini:

Tabel 4.1 Matrix Skenario dan Hasil Pengujian Fungsional Sistem

No.	Skenario Pengujian	Hasil Ekspektasi	Status (Berhasil/Gagal)
1.	Menjalankan program tebak angka dan menyimpan skor (Menu 1)	Skor berhasil terhitung dan masuk ke sistem.	Berhasil
2.	Menampilkan papan peringkat leaderboard (Menu 2)	Menampilkan daftar pemain berurutan dari skor tertinggi.	Berhasil

3.	Melakukan pencarian data nama pemain (Menu 3)	Nama ditemukan secara fleksibel meskipun huruf besar-kecil berbeda.	Berhasil
4.	Memperbarui skor pemain lama (Menu 4)	Angka skor ter-update dengan nilai input baru.	Berhasil
5.	Menghapus data pemain tertentu (Menu 5)	Data terhapus dari daftar utama dan masuk ke stack riwayat.	Berhasil
6.	Memeriksa riwayat log perubahan data (Menu 6)	Log terhapus tampil sesuai dengan urutan tumpukan LIFO.	Berhasil

4.2 Kelebihan Sistem

Berdasarkan hasil analisis perancangan dan fase pengujian fungsional yang telah dilaksanakan, aplikasi “Leaderboard Game Tebak Angka” yang dikembangkan oleh Kelompok 12 memiliki beberapa keunggulan teknis sebagai berikut:

1. Persistensi Data yang Stabil dan Otomatis.

Aplikasi ini memiliki keunggulan dalam menjaga keberadaan data skor pemain agar tidak hilang saat program dimatikan (*data persistence*). Melalui integrasi modul *File Handling* CSV (*load_data* dan *save_data*), setiap kali pemain menyelesaikan permainan, melakukan pembaruan skor, atau menghapus entri data, rname akan langsung melakukan sinkronisasi otomatis ke dalam penyimpanan fisik berkas *ZtugasAkhir/data.csv*.

2. Mekanisme Pertahanan Program yang Kuat (*Robustness*).

Aplikasi dilengkapi dengan rname penanganan galat (*error handling*) berlapis yang sangat andal. Penggunaan blok *try-except ValueError* pada menu input tebakan angka dan input skor baru berhasil mengisolasi kesalahan ketik dari pengguna (*human error*), seperti memasukkan karakter alfabet pada input numerik, sehingga program terhindar dari rname berhenti paksa (*crash/force close*). Selain itu, adanya penanganan *runtime exception* umum pada fungsi penulisan dokumen mampu mengamankan aplikasi dari kegagalan akibat pembatasan hak akses berkas

rname.

3. Optimalisasi Struktur Data yang Tepat Guna.

Sistem berhasil mengawinkan dua konsep struktur data secara harmonis sesuai kebutuhan fiturnya. Penggunaan Python List sebagai array dinamis mempermudah alokasi memori yang adaptif terhadap pertumbuhan data pemain. Sementara itu, pemanfaatan struktur data Stack (Tumpukan) dengan memanfaatkan metode penelusuran terbalik (`reversed()`) terbukti sangat ideal dan akurat dalam menyimulasikan log aktivitas penghapusan berbasis asas LIFO (Last In First Out).

4. Kualitas Pengalaman Pengguna (*User Experience*) yang Estetik.

Meskipun beroperasi di lingkungan teks murni (CLI), aplikasi ini memiliki nilai tambah dari segi estetika visual antarmuka. Dengan memanfaatkan konfigurasi tema warna dari *ANSI Escape Codes* (seperti PINK, YELLOW, CYAN), fitur pembersihan rnam berkala melalui `clear_screen()`, rata rname otomatis memakai fungsi `.center()`, serta pemberian rnament mahkota (👑) khusus untuk peringkat pertama pada *leaderboard*, aplikasi menjadi jauh lebih interaktif dan menarik untuk digunakan.

4.3 Kekurangan Sistem

Di samping kelebihan-kelebihan teknis yang dimiliki, terdapat beberapa aspek kelemahan pada struktur algoritma dan logika program yang perlu dievaluasi:

1. Kompleksitas Waktu Operasi Pencarian Linear yang Kurang Efisien

Algoritma pencarian, pembaruan, hingga penghapusan data pada aplikasi ini secara keseluruhan masih bergantung pada algoritma Linear Search (Pencarian Sekuensial) melalui perulangan `for` pemain in `players`:. Algoritma ini memiliki kompleksitas waktu kasus terburuk (*worst-case time complexity*) sebesar $O(n)$, di mana waktu komputasi akan meningkat secara linear berbanding lurus dengan jumlah data. Jika jumlah data pemain di dalam file CSV telah mencapai ribuan entri, proses pelacakan nama akan terasa melambat.

2. Sifat Peka Huruf (*Case-Sensitive*) yang Terlalu Kaku pada Manajemen Data

Tidak diadopsinya fungsi penyeragaman huruf seperti `.lower()` pada fungsi `cari_pemain()`, `update_skor()`, dan `hapus_pemain()` menyebabkan Tunggu pencarian dan eksekusi data berjalan sangat kaku. Pengguna dituntut untuk

mengingat dan mengetik susunan huruf besar serta huruf kecil secara mutlak sesuai dengan data asli. Kesalahan kecil seperti mengetik nama “Tungg” untuk mencari data “Nayla” akan langsung menyebabkan Tunga mengeluarkan pesan bahwa data tidak ditemukan, sehingga berpotensi menurunkan Tungal keramahan pengguna (*user-friendliness*).

3. Beban Proses I/O yang Tinggi Akibat Metode Overwrite File

Fungsi `save_data()` bekerja dengan mode akses berkas ‘w’ (*write*), yang berarti setiap kali terjadi perubahan sekecil apa pun pada data memori, Tunga akan menulis ulang (menimpa) seluruh isi berkas CSV dari baris pertama. Metode ini kurang efisien dari segi penggunaan sumber daya penyimpanan jika dibandingkan dengan metode *append* selektif, karena memaksa Tungal23 melakukan operasi baca-tulis (*I/O operations*) skala penuh untuk setiap Tunga satu data Tungal.

4.4 Keterbatasan Sistem

Keterbatasan sistem mendefinisikan batasan ruang lingkup operasional, lingkungan eksekusi, serta arsitektur bawaan yang tidak dapat dilampaui oleh aplikasi dalam kondisinya saat ini:

1. Keterbatasan Lingkungan Antarmuka Pengguna (Terbatas pada CLI)

Sistem ini sepenuhnya beroperasi secara eksklusif pada lingkungan *Command Line Interface* (CLI) atau terminal teks murni. Aplikasi belum mendukung antarmuka grafis atau *Graphical User Interface* (GUI) yang berbasis jendela (*window*), tombol, ataupun menu klik, sehingga cakupan distribusinya hanya terbatas untuk pengguna yang terbiasa dengan pengoperasian konsol.

2. Keamanan Basis Data Lokal yang Rendah

Berkas basis data eksternal `data.csv` disimpan dalam format teks mentah terpisah koma (*plain text*) yang tidak terlindungi. Ketiadaan mekanisme enkripsi atau fungsi pencacahan (*hashing*) pada file CSV membuat database ini sangat rentan terhadap manipulasi ilegal dari luar program. Siapa pun dapat membuka folder `ZTugasAkhir` dan mengubah nilai skor pemain secara bebas menggunakan aplikasi penyunting teks biasa seperti Notepad atau Microsoft Excel.

3. Sifat Riwayat Log Penghapusan (Stack) yang Volatil (Sementara)

Struktur data `history_stack` yang bertindak sebagai penyimpan log riwayat penghapusan data bersifat volatil atau transien. Karena variabel tumpukan tersebut

hanya dialokasikan di dalam memori RAM komputer dan tidak ikut disinkronkan ke dalam penyimpanan berkas permanen CSV, seluruh log riwayat perubahan LIFO tersebut akan langsung musnah terhapus secara permanen seketika pengguna memilih menu 0 untuk keluar dari aplikasi atau saat program dihentikan.

BAB V. KENDALA DAN SOLUSI

Selama proses pengembangan aplikasi, kelompok kami menghadapi beberapa kendala teknis dalam implementasi kode. Berikut adalah rincian kendala beserta solusi yang diterapkan:

No.	Kendala / Masalah Error	Solusi yang Diterapkan
1.	Terjadi <code>FileNotFoundError</code> atau Error Direktori: Program mengalami kegagalan saat pertama kali dijalankan karena folder <code>ZTugasAkhir</code> belum tersedia di komputer pengguna.	Penerapan <code>os.path.exists</code> : Di dalam fungsi <code>load_data()</code> , ditambahkan validasi otomatis menggunakan library <code>os</code> . Jika folder atau file CSV belum ada, sistem akan langsung membuat direktori dan file baru secara otomatis dengan <i>header</i> yang sesuai.
2.	Aplikasi Crash Akibat Salah Input (<code>ValueError</code>): Ketika pengguna tidak sengaja memasukkan karakter huruf pada input tebakan angka (Menu 1) atau input skor baru (Menu 4), program langsung berhenti (<i>force close</i>).	Implementasi <code>try-except</code> Block: Memasang mekanisme penanganan eksepsi <code>try ... except ValueError</code> di bagian input angka. Jika input yang dimasukkan bukan integer, program akan menampilkan pesan peringatan yang rapi tanpa merusak jalannya aplikasi.
3.	Potensi Inkonsistensi Tipe Data Skor: Pada beberapa kondisi pengujian, skor yang dibaca dari file CSV terkadang berubah menjadi tipe data string, sehingga mempersulit kalkulasi matematika selanjutnya.	Validasi Casting Integer: Pada fungsi <code>save_data()</code> , ditambahkan iterasi pembersihan data (<code>p['skor'] = int(p['skor'])</code>) untuk memastikan bahwa seluruh nilai skor dikonversi kembali menjadi integer murni secara konsisten sebelum disimpan ke dokumen.1.

BAB VI. PEMBAGIAN TUGAS ANGGOTA

No.	Nama	NIM	Pembagian Tugas
1.	Muhammad Rezki Khaidir	J0403251127	Bertanggung jawab atas perancangan struktur data tumpukan log (Stack History LIFO), pembuatan fitur penghapusan data pemain (Delete Data), pembuatan diagram alir (Flowchart), serta perancangan skenario pengujian sistem.
2.	Najla Septian Putri	J0403251010	Bertanggung jawab atas pembuatan arsitektur Menu Utama, Fitur Game Tebak Angka, Manajemen Menu Penambahan Skor (Create Data), dan Integrasi File Handling CSV (Save/Load Data).
3.	Nayla Aqila Argia	J0403251142	Bertanggung jawab atas pengerjaan komponen pencarian pemain (Linear Searching), pembaruan data (Update Score), penataan logika fungsi pengurutan (Sorting Leaderboard)

BAB VII. KESIMPULAN DAN SARAN

7.1 Kesimpulan

Berdasarkan seluruh tahapan perancangan, implementasi, hingga pengujian yang telah dilakukan pada proyek tugas akhir ini, dapat disimpulkan bahwa aplikasi *Leaderboard Game* berhasil dikembangkan secara fungsional sebagai media pencatatan skor yang sistematis. Implementasi struktur data dan algoritma dalam bahasa pemrograman Python terbukti mampu mengelola aliran data pemain, mulai dari proses penyimpanan instan (*save data*) hingga mekanisme pencarian (*searching*) skor secara responsif. Keberhasilan proyek ini sekaligus merepresentasikan penerapan praktis dari teori mata kuliah Algoritma dan Struktur Data Semester 2 ke dalam sebuah solusi perangkat lunak yang riil.

Selama proses pengembangan, sistem ini juga telah mengintegrasikan mekanisme penanganan *error* yang adaptif (*robustness*). Penerapan validasi otomatis `os.path.exists` pada fungsi pemuatan data berhasil mengatasi kendala kegagalan direktori (`FileNotFoundException`), sementara blok eksepsi `try-except ValueError` efektif dalam mencegah terjadinya kerusakan sistem (*crash/force close*) akibat kesalahan input pengguna. Selain itu, inkonsistensi tipe data saat pembacaan berkas eksternal berhasil dimitigasi secara konsisten melalui penataan *casting* integer pada fungsi penyimpanan. Dengan demikian, aplikasi ini tidak hanya berfungsi dengan baik dari segi fitur, tetapi juga memiliki ketahanan yang tinggi terhadap kegagalan operasional (*runtime errors*).

7.2 Saran

Meskipun aplikasi *Leaderboard Game* ini telah memenuhi spesifikasi fungsional utama yang ditargetkan, masih terdapat beberapa ruang pengembangan yang dapat dioptimalkan lebih lanjut. Oleh karena itu, diajukan beberapa saran strategis bagi pengembangan sistem ke depannya:

1. Optimalisasi Struktur Data Tingkat Lanjut

Untuk penelitian atau pengembangan selanjutnya, disarankan untuk mengganti struktur data penyimpanan sementara atau array linear standar dengan struktur data pohon yang seimbang, seperti *Balanced Binary Search*

Tree (Pohon AVL) atau konsep *Heap*. Penerapan struktur data ini diproyeksikan dapat meningkatkan efisiensi waktu pemrosesan (*time complexity*) secara signifikan saat melakukan pengurutan (*sorting*) dan penentuan peringkat skor dalam skala data yang jauh lebih besar.

2. Pengembangan Antarmuka dan Keamanan Data

Aplikasi ini dapat dikembangkan lebih lanjut dengan memigrasikan antarmuka berbasis teks (*Command Line Interface/CLI*) menuju antarmuka grafis yang interaktif (*Graphical User Interface/GUI*) menggunakan pustaka seperti Tkinter atau PyQt agar lebih ramah pengguna (*user-friendly*). Selain itu, aspek keamanan data pada berkas CSV dapat ditingkatkan dengan menerapkan algoritma kriptografi atau enkripsi sederhana guna mencegah manipulasi skor secara ilegal oleh pihak luar.

DAFTAR PUSTAKA

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, Cambridge, MA, USA: MIT Press, 2022.
- [2] M. T. Goodrich, R. Tamassia and M. H. Goldwasser, Data Structures and Algorithms in Python, Hoboken, NJ, USA: John Wiley & Sons, 2013.
- [3] B. Miller and D. Ranum, Problem Solving with Algorithms and Data Structures Using Python, Hopkins, MN, USA: Franklin, Beedle & Associates, 2011.
- [4] M. Lutz, Learning Python, Sebastopol, CA, USA: O'Reilly Media, 2013.
- [5] R. Stephens, Beginning Software Engineering, 2nd ed., Indianapolis, IN, USA: John Wiley & Sons, 2022.
- [6] A. Downey, Think Python: How to Think Like a Computer Scientist, 2nd ed., Sebastopol, CA, USA: O'Reilly Media, 2015.
- [7] G. van Rossum, 2023. [Online]. Available: <https://docs.python.org/3/library/>.

LAMPIRAN

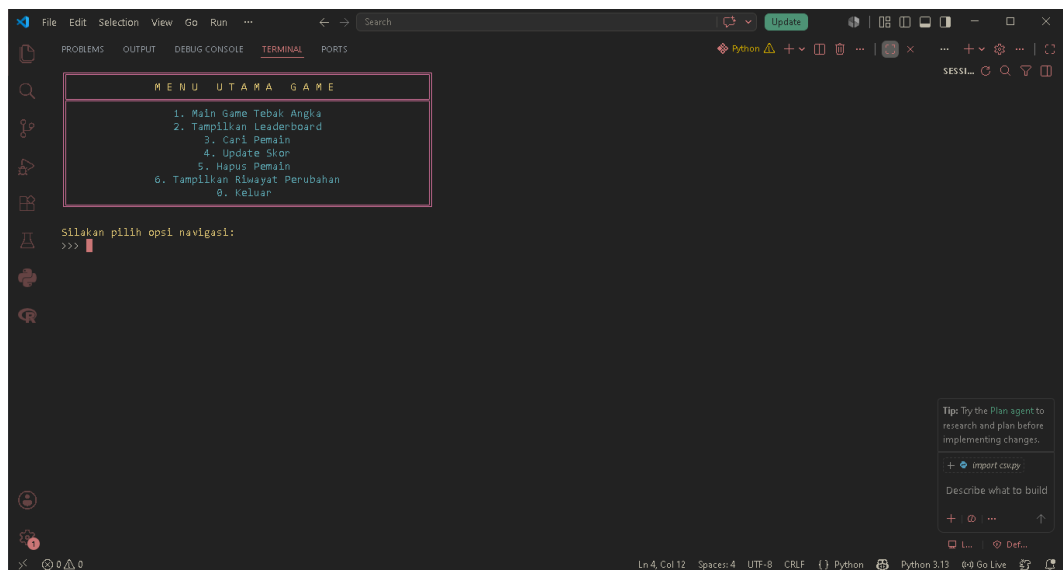
Lampiran A. Link Github

Seluruh kode sumber (*source code*), berkas data, dokumen konfigurasi, dan riwayat pengembangan proyek aplikasi "Leaderboard Game" ini telah diunggah dan dikelola secara publik pada repositori GitHub melalui tautan berikut:

- Tautan Repositori: https://github.com/naymiwa/ASD_B_Kelompok12

Lampiran B. Screenshot Program

1. Tampilan Menu Utama: Menampilkan visualisasi antarmuka CLI berbentuk kotak dengan dekorasi warna-warni dari *ANSI Escape Codes* serta pilihan navigasi menu 1 sampai 6 dan menu 0.

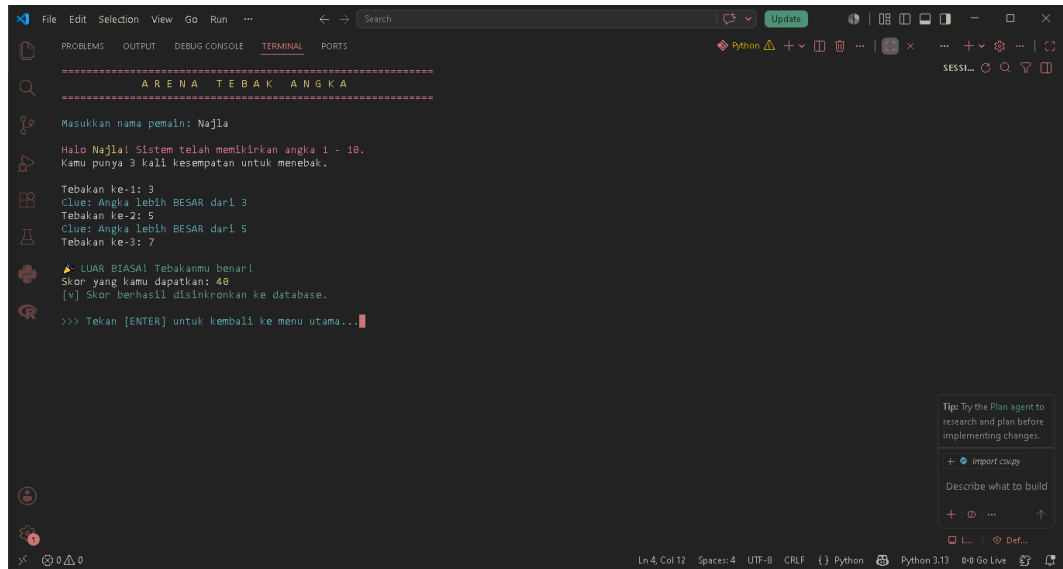


```

MENU UTAMA GAME
1. Main Game Tebak Angka
2. Tampilkan Leaderboard
3. Cari Pemain
4. Update Skor
5. Hapus Pemain
6. Tampilkan Riwayat Perubahan
0. Keluar

Silakan pilih opsi navigasi:
>>> 
```

2. Sesi Arena Tebak Angka (Menu 1): Menampilkan proses input nama pemain, interaksi tebakan angka beserta petunjuk (*clue*) "Lebih BESAR" atau "Lebih KECIL", hingga notifikasi keberhasilan sinkronisasi skor ke database CSV.



```
=====
A R E N A   T E B A K   A N G K A
=====

Masukkan nama pemain: Najla

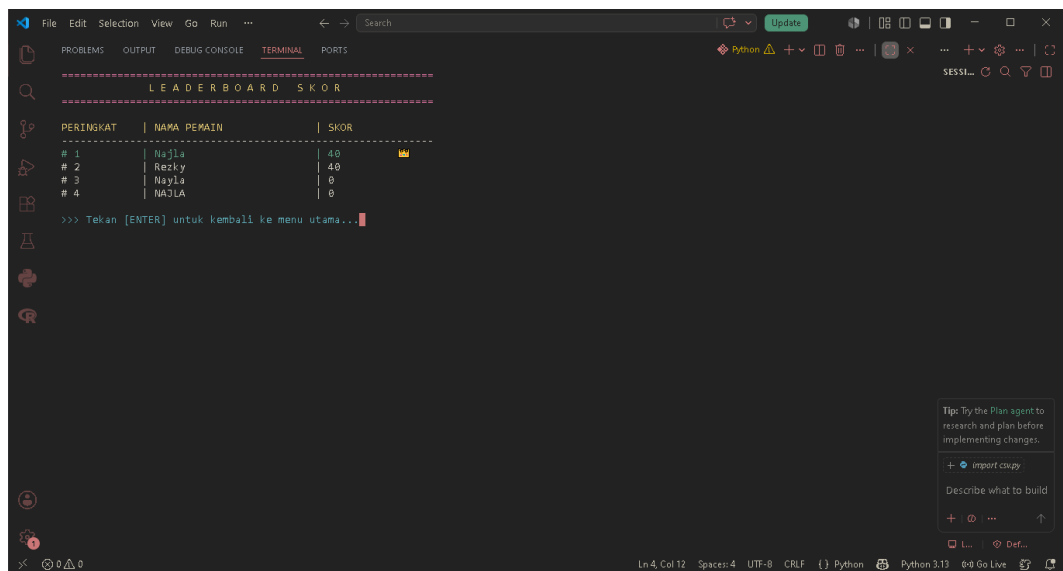
Halo Najla! Sistem telah memikirkan angka 1 - 10.
Kamu punya 3 kali kesempatan untuk menebak.

Tebakan ke-1: 3
Clue: Angka lebih BESAR dari 3
Tebakan ke-2: 5
Clue: Angka lebih BESAR dari 5
Tebakan ke-3: 7

👉 LUAR BIASAL Tebakanmu benar!
Skor yang kamu dapatkan: 40
[V] Skor berhasil disinkronkan ke database.

>>> Tekan [ENTER] untuk kembali ke menu utama...
```

3. Tampilan Leaderboard Skor (Menu 2): Menampilkan tabel peringkat data pemain yang telah terurut secara menurun (*descending*) dengan *highlight* khusus ikon mahkota (👑) pada peringkat pertama.



```
=====
LEADERBOARD SKOR
=====

PERINGKAT | NAMA PEMAIN | SKOR
-----|-----|-----
# 1       | Najla       | 40    👑
# 2       | Rezky      | 40
# 3       | Nayla      | 0
# 4       | NAJLA      | 0

>>> Tekan [ENTER] untuk kembali ke menu utama...
```

4. Fitur Cari, Update, dan Hapus (Menu 3, 4, dan 5): Menampilkan simulasi pelacakan nama secara *case-sensitive*, proses manipulasi skor baru, serta konfirmasi penghapusan data.

```
File Edit Selection View Go Run ... Search
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
=====
PENCARIAN PEMAIN
=====
Masukkan nama pemain yang ingin dicari: Najla
=====
Ditemukan 1 data yang cocok:
=====
1. Najla - Skor: 40
=====
>>> Tekan [ENTER] untuk kembali ke menu utama...
```

Tip: Try the Plan agent to research and plan before implementing changes.

+ import copy

Describe what to build

+ ...

Ln 4, Col 12 Spaces: 4 UTF-8 CRLF Python Python 3.13 Go Live

```
File Edit Selection View Go Run ... Search
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
=====
UPDATE SKOR PEMAIN
=====
Masukkan nama pemain secara spesifik: NAJLA
Masukkan skor baru untuk NAJLA: 100
[v] Skor untuk NAJLA berhasil diperbarui menjadi 100.
>>> Tekan [ENTER] untuk kembali ke menu utama...
```

Tip: Try the Plan agent to research and plan before implementing changes.

+ import copy

Describe what to build

+ ...

Ln 4, Col 12 Spaces: 4 UTF-8 CRLF Python Python 3.13 Go Live

```
File Edit Selection View Go Run ... Search
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
=====
HAPUS DATA PEMAIN
=====
Masukkan nama pemain yang akan dihapus: Najla
Yakin ingin menghapus Najla? (y/n): y
[v] Data berhasil dihapus dan disimpan ke riwayat.
>>> Tekan [ENTER] untuk kembali ke menu utama...
```

Tip: Try the Plan agent to research and plan before implementing changes.

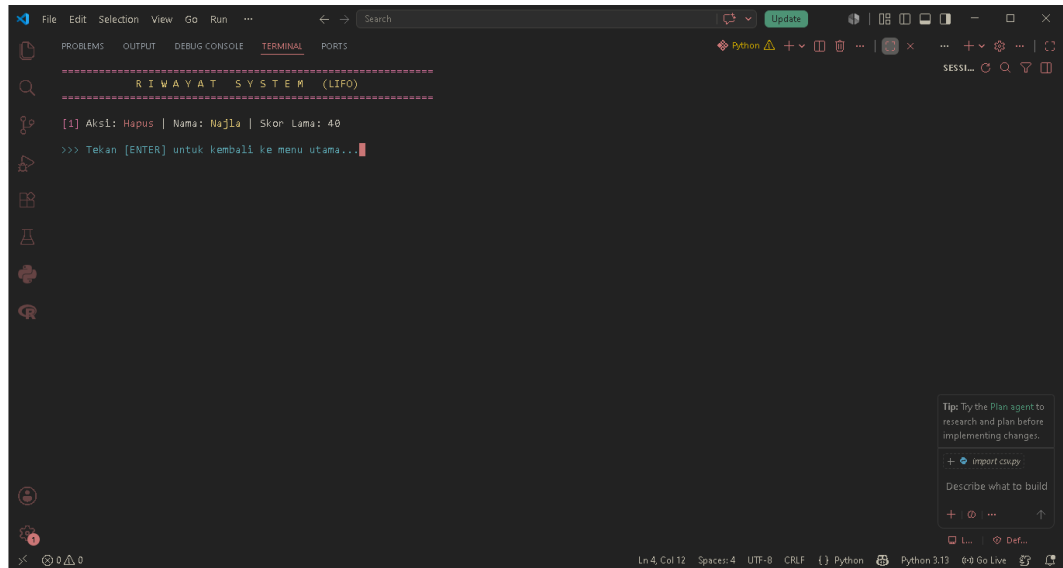
+ import copy

Describe what to build

+ ...

Ln 4, Col 12 Spaces: 4 UTF-8 CRLF Python Python 3.13 Go Live

5. Tampilan Riwayat System LIFO (Menu 6): Menampilkan urutan log data yang berhasil dihapus dari sistem dengan urutan tumpukan terbalik (*Last In First Out*).



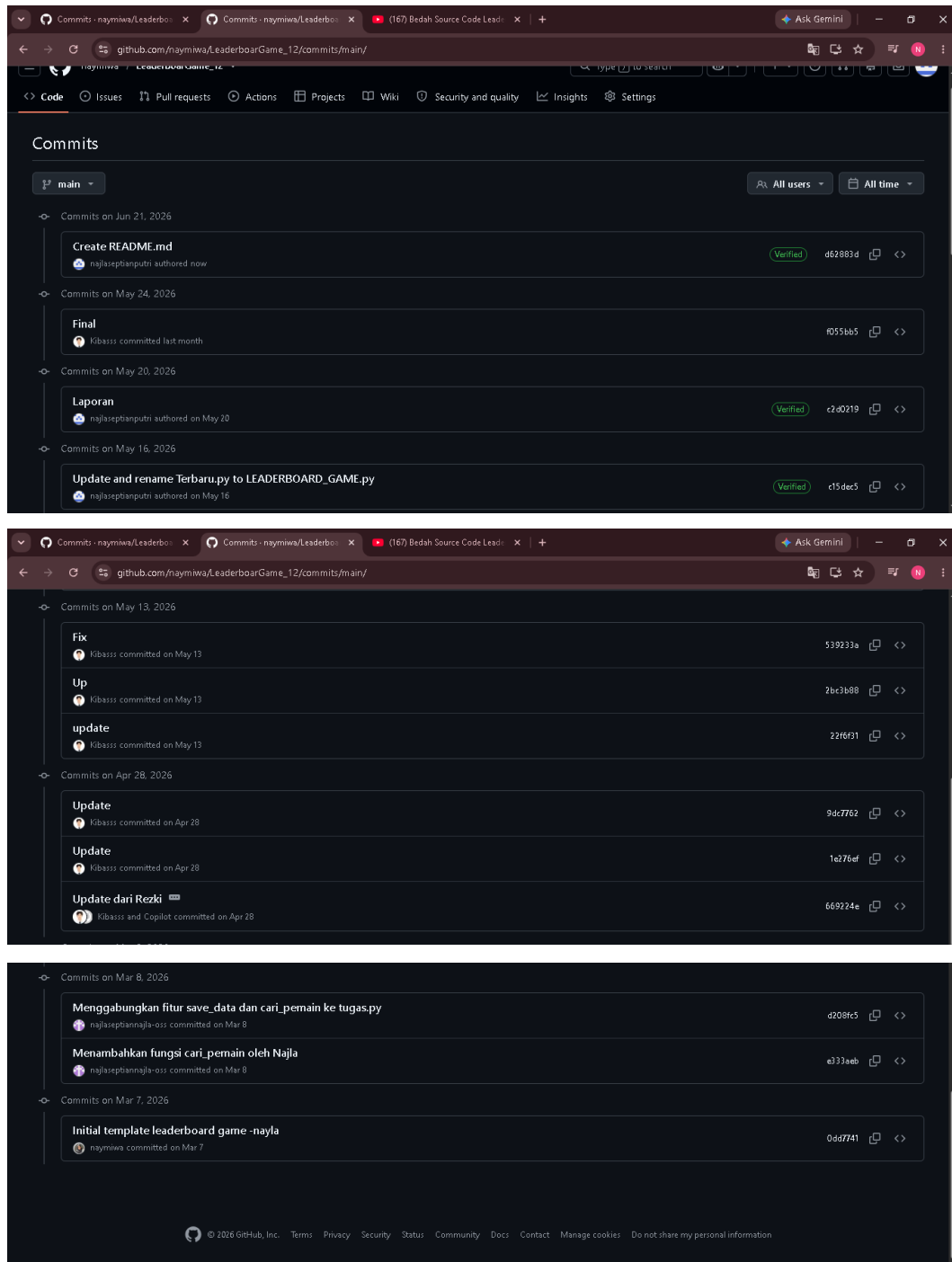
Lapiran C. Struktur Folder Proyek

Berikut adalah visualisasi pohon struktur direktori dan tata letak berkas komponen penyusun aplikasi "Leaderboard Game Tebak Angka" di dalam komputer lokal:



Lampiran D. Commit History Github

Riwayat *commit* di bawah ini mencerminkan kontribusi aktif, kolaborasi, dan linimasa pengerjaan kode program yang dilakukan oleh masing-masing anggota Kelompok 12 (Najla, Nayla, dan Rezki) secara berkala pada repositori GitHub:



Lapiran E. Source Code Utama

Berikut adalah dokumentasi baris kode program utuh dari file LEADERBOARD_GAME.py yang digunakan untuk mengoperasikan seluruh sistem aplikasi:

```
import csv
import os
import random
import time

# =====
# NAYLA: KONFIGURASI TEMA WARNA & UI (ANSI ESCAPE CODES)
# Digunakan untuk memberikan warna pada teks di terminal
# =====
PINK = '\033[95m'
YELLOW = '\033[93m'
CYAN = '\033[96m'
GREEN = '\033[92m'
RED = '\033[91m'
RESET = '\033[0m' # Mengembalikan warna terminal ke default
LEBAR_TERMINAL = 60

# =====
# DEKLARASI VARIABEL GLOBAL & STRUKTUR DATA
# =====
FILE_NAME = "ZTugasAkhir/data.csv"

# 1. LIST (Array Dinamis)
players = []

# 2. STACK (Tumpukan - LIFO)
# Digunakan untuk menyimpan riwayat penghapusan data.
history_stack = []

# =====
# NAJLA: FUNGSI BANTUAN UX (USER EXPERIENCE)
# =====
def clear_screen():
    """Membersihkan layar terminal agar UI terlihat rapi saat pindah menu."""
    os.system('cls' if os.name == 'nt' else 'clear')

def jeda_kembali():
    """Memberikan jeda bagi user untuk membaca output sebelum kembali ke menu."""
    input(f"\n{CYAN}>>> Tekan [ENTER] untuk kembali ke menu utama...{RESET}")

def cetak_judul(judul):
    """Mencetak judul menu dengan format border yang posisinya persis di tengah."""
    clear_screen()
    print(PINK + "=" * LEBAR_TERMINAL + RESET)
    print(YELLOW + judul.center(LEBAR_TERMINAL) + RESET) #
    .center() untuk rata tengah presisi
```

```

    print(PINK + "=" * LEBAR_TERMINAL + RESET + "\n")

# =====
# NAYLA: LOAD DATA (Membaca file CSV ke List)
# =====
def load_data():
    players.clear() # Bersihkan list sebelum memuat data baru

    # Validasi dan pembuatan folder jika belum ada agar terhindar
    dari FileNotFoundError
    if not os.path.exists("ZTugasAkhir"):
        os.makedirs("ZTugasAkhir")

    # Jika file CSV belum ada, buat file baru beserta headernya
    if not os.path.exists(FILE_NAME):
        with open(FILE_NAME, "w", newline="") as file:
            writer = csv.writer(file)
            writer.writerow(["nama", "skor"])
        return

    # Membaca data dari CSV dan memasukkannya ke dalam list of
    dictionaries 'players'
    with open(FILE_NAME, "r") as file:
        reader = csv.DictReader(file)
        for row in reader:
            players.append({
                "nama": row["nama"],
                "skor": int(row["skor"]) # Konversi skor ke integer
            })

# =====
# NAJLA: SAVE DATA (File Handling / Menulis ke CSV)
# =====
def save_data():
    try:
        with open(FILE_NAME, mode='w', newline='') as file:
            fieldnames = ['nama', 'skor']
            writer = csv.DictWriter(file, fieldnames=fieldnames)

            writer.writeheader() # Menulis ulang header kolom

            # Memastikan seluruh nilai skor adalah integer murni
            sebelum disimpan
            for p in players:
                p['skor'] = int(p['skor'])

            writer.writerows(players) # Menulis seluruh isi list
            'players' ke CSV
    except Exception as e:
        # Menangani error jika file sedang dibuka oleh program lain
        atau hak akses ditolak
        print(f"{RED}[!] Kesalahan: Gagal menyimpan data. Alasan:
        {e}{RESET}")

# =====
# NAYLA & REZKI: PLAY GAME
# =====
def play_game():

```

```

    cetak_judul("A R E N A   T E B A K   A N G K A")

    player_name = input(f"{CYAN}Masukkan nama pemain:
{RESET}").strip()
    if player_name == "":
        print(f"{RED}[!] Nama tidak boleh kosong.{RESET}")
        jeda_kembali()
        return

    print(f"\n{PINK}Halo {YELLOW}{player_name}{PINK}! Sistem telah
memikirkan angka 1 - 10.{RESET}")
    print("Kamu punya 3 kali kesempatan untuk menebak.\n")

    # Menghasilkan angka acak dari 1 hingga 10
    angka_rahasia = random.randint(1, 10)
    kesempatan = 3
    skor = 0

    for i in range(1, kesempatan + 1):
        try:
            tebakan = int(input(f"Tebakan ke-{i}: "))
        except ValueError:
            # Mencegah program crash (force close) jika user
menginput huruf/symbol
            print(f"{RED}[!] Masukkan angka yang valid!{RESET}")
            continue

        if tebakan == angka_rahasia:
            # Algoritma perhitungan skor: semakin cepat menebak,
skor semakin tinggi
            skor = 100 - (i - 1) * 30
            print(f"\n{GREEN}🎉 LUAR BIASA! Tebakanmu
benar!{RESET}")
            print(f"Skor yang kamu dapatkan:
{YELLOW}{skor}{RESET}")
            break
        else:
            if i < kesempatan:
                # Memberikan petunjuk (clue) navigasi tebakan
                if tebakan < angka_rahasia:
                    print(f"{CYAN}Clue: Angka lebih BESAR dari
{tebakan}{RESET}")
                else:
                    print(f"{CYAN}Clue: Angka lebih KECIL dari
{tebakan}{RESET}")
            else:
                print(f"\n{RED}Game Over! Angka yang benar adalah:
{angka_rahasia}{RESET}")
                print(f"Skor kamu: {YELLOW}0{RESET}")

    # Memanggil modul terpisah untuk menyimpan skor
    simpan_skor(player_name, skor)
    jeda_kembali()

# =====
# NAJLA: SIMPAN SKOR
# =====
def simpan_skor(nama, skor):
    """Menambahkan data pemain baru ke list global dan trigger

```



```

save_data."""
    players.append({"nama": nama, "skor": skor})
    save_data()
    print(f"{GREEN}[v]      Skor      berhasil      disinkronkan      ke
database.{RESET}")

# =====
# REZKI: TAMPILKAN LEADERBOARD (Implementasi Algoritma SORTING)
# =====
def tampilkan_leaderboard():
    cetak_judul("L E A D E R B O A R D      S K O R")

    if not players:
        print(f"{CYAN}Belum      ada      data      pemain      di
leaderboard.{RESET}".center(LEBAR_TERMINAL))
    else:
        # ALGORITMA SORTING:
        # Mengurutkan list 'players' berdasarkan *key* 'skor'.
        # reverse=True digunakan agar urutannya Descending (Skor
        terbesar di atas).
        sorted_players = sorted(players, key=lambda x: x['skor'],
reverse=True)

        # Formatting output menjadi bentuk tabel yang rapi
        print(f"{YELLOW}{'PERINGKAT':<12} | {'NAMA PEMAIN':<25} |
{'SKOR':<10}{RESET}")
        print("-" * LEBAR_TERMINAL)
        for idx, p in enumerate(sorted_players, 1):
            if idx == 1:
                # Memberikan highlight khusus (ikon mahkota) untuk
                Peringkat 1
                print(f"{GREEN}{'#'+str(idx):<12}      |
{p['nama']:<25} | {p['skor']:<10} 🏆{RESET}")
            else:
                print(f"{'#'+str(idx):<12} | {p['nama']:<25} |
{p['skor']:<10}")
            jeda_kembali()

# =====
# NAJLA: CARI PEMAIN (Implementasi Algoritma SEARCHING)
# =====
def cari_pemain():
    cetak_judul("P E N C A R I A N      P E M A I N")

    nama_dicari = input(f"{CYAN}Masukkan nama pemain yang ingin
dicari: {RESET}").strip()
    hasil_pencarian = []

    # HAPUS fungsi .lower() di sini
    for pemain in players:
        if nama_dicari in pemain['nama']:
            hasil_pencarian.append(pemain)

    print("\n" + "-" * LEBAR_TERMINAL)
    if hasil_pencarian:
        print(f"{GREEN}Ditemukan {len(hasil_pencarian)} data yang
cocok:{RESET}\n")
        for idx, p in enumerate(hasil_pencarian, 1):
            print(f"      {idx}. {YELLOW}{p['nama']}{RESET} - Skor:

```

```

{CYAN}{p['skor']}{RESET}")
    else:
        print(f"{RED}Data dengan nama '{nama_dicari}' tidak
ditemukan.{RESET}")
        print("-" * LEBAR_TERMINAL)
        jeda_kembali()

# =====
# REZKI: UPDATE SKOR
# =====
def update_skor():
    cetak_judul("U P D A T E   S K O R   P E M A I N")

    nama_dicari = input(f"{CYAN}Masukkan nama pemain secara
spesifik: {RESET}").strip()

    # HAPUS fungsi .lower() di sini agar exact match (harus sama
persis)
    for pemain in players:
        if pemain['nama'] == nama_dicari:
            try:
                skor_baru = int(input(f"Masukkan skor baru untuk
{YELLOW}{pemain['nama']}{RESET}: "))
                pemain['skor'] = skor_baru
                save_data() # Sinkronisasi perubahan ke CSV
                print(f"\n{GREEN}[v] Skor untuk {pemain['nama']}
berhasil diperbarui menjadi {skor_baru}.{RESET}")
            except ValueError:
                print(f"\n{RED}[] Skor harus berupa angka. Update
dibatalkan.{RESET}")
                jeda_kembali()
            return

    print(f"\n{RED}Pemain dengan nama '{nama_dicari}' tidak
ditemukan. Update dibatalkan.{RESET}")
    jeda_kembali()

# =====
# NAYLA: HAPUS PEMAIN & PUSH KE STACK RIWAYAT
# =====
def hapus_pemain():
    cetak_judul("H A P U S   D A T A   P E M A I N")

    nama = input(f"{CYAN}Masukkan nama pemain yang akan dihapus:
{RESET}").strip()

    # HAPUS fungsi .lower() di sini
    for p in players:
        if p["nama"] == nama:
            konfirmasi = input(f"Yakin ingin menghapus
{YELLOW}{p['nama']}{RESET}? (y/n): ")
            if konfirmasi.lower() == 'y':
                history_stack.append(("Hapus", p.copy()))
                players.remove(p) # Hapus dari list utama
                save_data() # Update file CSV
                print(f"\n{GREEN}[v] Data berhasil dihapus dan
disimpan ke riwayat.{RESET}")
            else:
                print(f"\n{CYAN}[-]                                Penghapusan

```

```

dibatalkan.{RESET}")
    jeda_kembali()
    return

    print(f"\n{RED}[!] Pemain tidak ditemukan.{RESET}")
    jeda_kembali()

# =====
# REZKI: TAMPILKAN RIWAYAT (Implementasi Struktur Data STACK)
# =====
def tampilkan_riwayat():
    cetak_judul("R I W A Y A T      S Y S T E M      (LIFO)")

    if not history_stack:
        print(f"{CYAN}Belum      ada      riwayat      penghapusan
data.{RESET}".center(LEBAR_TERMINAL))
    else:
        # Menampilkan data Stack menggunakan reversed()
        # untuk menyimulasikan sifat LIFO (Last In First Out).
        for idx, (aksi, data) in enumerate(reversed(history_stack),
1):
            print(f"{PINK}[{idx}]{RESET} Aksi: {RED}{aksi}{RESET}
| Nama: {YELLOW}{data['nama']}{RESET} | Skor Lama: {data['skor']}")
            jeda_kembali()

# =====
# MAIN MENU & ENTRY POINT
# =====
def menu():
    """Merender antarmuka menu utama berbentuk kotak."""
    clear_screen()
    print(PINK + "┌" + "=" * (LEBAR_TERMINAL - 2) + "┐" + RESET)
    print(PINK + "│" + YELLOW + "M E N U      U T A M A      G A M
E".center(LEBAR_TERMINAL - 2) + PINK + "│" + RESET)
    print(PINK + "└" + "=" * (LEBAR_TERMINAL - 2) + "┘" + RESET)

    menu_items = [
        "1. Main Game Tebak Angka",
        "2. Tampilkan Leaderboard",
        "3. Cari Pemain",
        "4. Update Skor",
        "5. Hapus Pemain",
        "6. Tampilkan Riwayat Perubahan",
        "0. Keluar"
    ]

    for item in menu_items:
        print(PINK + "│" + CYAN + item.center(LEBAR_TERMINAL - 2)
+ PINK + "│" + RESET)

    print(PINK + "└" + "=" * (LEBAR_TERMINAL - 2) + "┘" + RESET)

def main():
    """Fungsi utama yang menjalankan sistem navigasi program."""
    load_data() # Memuat data CSV ke memori saat program pertama
kali dijalankan

    while True:
        menu()

```

```

        print(f"\n{YELLOW}Silakan pilih opsi navigasi:{RESET}")
        pilih = input(">>> ").strip()

        # Percabangan logika untuk mengarahkan pilihan user ke
fungsi yang sesuai
        if pilih == "1":
            play_game()
        elif pilih == "2":
            tampilkan_leaderboard()
        elif pilih == "3":
            cari_pemain()
        elif pilih == "4":
            update_skor()
        elif pilih == "5":
            hapus_pemain()
        elif pilih == "6":
            tampilkan_riwayat()
        elif pilih == "0":
            # Keluar dari program (Exit Point)
            clear_screen()
            print(f"{PINK}=" * LEBAR_TERMINAL + RESET)
            print(f"{YELLOW}Terima kasih telah menggunakan sistem
Leaderboard Game!{RESET}".center(LEBAR_TERMINAL + 10))
            print(f"{PINK}=" * LEBAR_TERMINAL + RESET + "\n")
            break
        else:
            print(f"{RED}[!] Pilihan tidak valid. Silakan masukkan
angka 0-6.{RESET}")
            time.sleep(1.5) # Memberi jeda sebentar sebelum menu
dirender ulang

# Script entry point standar di Python
if __name__ == "__main__":
    main()

```